

Documentación Técnica - IRLGYM

1. Arquitectura del Proyecto

La aplicación sigue los principios de la **Clean Architecture** (Arquitectura Limpia), separando el código en cuatro capas principales para garantizar escalabilidad, testabilidad e independencia de frameworks:

- **Domain (Dominio):** Entidades e interfaces centrales. No depende de nada externo.
- **Data (Datos):** Implementación de repositorios (SQLite local y Firebase nube) y gestión de sincronización.
- **Application (Aplicación):** Casos de uso y lógica de negocio (Servicios).
- **Presentation (Presentación):** Componentes visuales de React Native/Expo, pantallas, navegación y estilos.
- **Infrastructure (Infraestructura):** Configuraciones de terceros (Firebase, SQLite).

2. Capa de Dominio (src/domain/)

Contiene las reglas de negocio más puras.

Entidades (entities/)

- **LibraryExercise.ts:** Define la estructura de un ejercicio (nombre, grupo muscular, si es favorito o personalizado).
- **SkillNode.ts:** Define los nodos del árbol de habilidades RPG (título, requisitos de XP, estado de desbloqueo).
- **User.ts:** Define el perfil del usuario (nombre, email, unidades de medida) y sus estadísticas globales (nivel, XP, racha).
- **Workout.ts:** Define un entrenamiento (Workout), las series (Set) y los registros individuales de cada ejercicio (WorkoutLog). Incluye propiedades como `isLocked` para rutinas de recompensa.

Interfaces de Repositorios (repositoriesInterface/)

- **IExerciseRepository.ts:** Contrato para operaciones CRUD sobre ejercicios de la biblioteca.
- **ILogRepository.ts:** Contrato para guardar y recuperar series/logs de entrenamiento.
- **IProgressRepository.ts:** Contrato para gestionar el progreso del usuario en el árbol de habilidades.
- **ISkillRepository.ts:** Contrato para leer los nodos disponibles en el árbol.
- **IUserProfileRepository.ts:** Contrato para la gestión del perfil de usuario.
- **IUserStatsRepository.ts:** Contrato para la gestión de estadísticas de XP y nivel.
- **IWorkoutRepository.ts:** Contrato para gestionar rutinas y plantillas de entrenamiento.

3. Capa de Datos (src/data/)

Implementa las interfaces del dominio para persistir datos. Se utiliza un sistema híbrido (Local First) para asegurar el funcionamiento offline.

repositories/index.ts

Punto central que exporta instancias de `HybridRepository`. Esta clase envoltorio recibe un repositorio local (SQLite) y uno remoto (Firebase). Al realizar una operación, primero la ejecuta en local y luego, si hay conexión, la sincroniza con la nube de forma transparente.

Repositorios SQLite (sqlite/)

- **SQLiteClient.ts** (Infraestructura): Singleton que mantiene la conexión a la base de datos `gym_rpg.db` y activa `PRAGMA foreign_keys = ON`.
- **sqlite.ts** (Infraestructura): Script de inicialización que crea las tablas y siembra los ejercicios base.
- **SQLiteExerciseRepository.ts:** CRUD local de ejercicios. Incluye `clearUserData()` para el borrado de cuenta.
- **SQLiteLogRepository.ts:** Guarda las series realizadas asociadas a entrenamientos.
- **SQLiteProgressRepository.ts:** Guarda el estado de cada nodo de habilidad para el usuario.
- **SQLiteSkillRepository.ts:** Repositorio de solo lectura para la estructura estática del árbol de habilidades.
- **SQLiteUserProfileRepository.ts:** Guarda los datos personales del usuario.
- **SQLiteUserStatsRepository.ts:** Guarda nivel, XP y rachas.
- **SQLiteWorkoutRepository.ts:** Guarda rutinas en progreso, completadas y plantillas.

Repositorios Firebase (firebase/)

Implementan los mismos contratos pero atacando a `Firestore`. Sirven como backup en la nube.

- **FirebaseExerciseRepository.ts**

- `FirebaseLogRepository.ts`
- `FirebaseProgressRepository.ts`
- `FirebaseSkillRepository.ts`
- `FirebaseUserProfileRepository.ts`
- `FirebaseUserStatsRepository.ts`
- `FirebaseWorkoutRepository.ts`

`syncManager.ts`

Lógica diseñada para sincronizar datos locales pendientes (`sync_status = 1`) con Firebase cuando la aplicación recupera la conexión a internet.

4. Capa de Aplicación (`src/application/service/`)

Contiene los casos de uso (Business Logic) orquestando entidades y repositorios.

- `AuthService.ts`: Gestiona el inicio de sesión, registro, cierre de sesión y el borrado atómico de cuenta (eliminando datos en cascada en SQLite y Firebase).
 - `ExerciseService.ts`: Lógica para filtrar ejercicios, marcar favoritos y calcular el historial de récords y volumen máximo.
 - `LogService.ts`: Validación y guardado de series de entrenamiento.
 - `ProgressService.ts`: Consulta el progreso general del usuario.
 - `SkillService.ts`: Núcleo RPG. Calcula si un nodo puede desbloquearse comprobando XP, otorga recompensas (rutinas especiales) y consume XP al desbloquear.
 - `UserProfileService.ts`: Gestión de actualizaciones de perfil.
 - `UserStatsService.ts`: Sistema de progresión. Suma XP tras cada entrenamiento y gestiona el algoritmo de subida de nivel.
 - `WorkoutService.ts`: Orquesta el flujo de un entrenamiento: crear desde plantilla, finalizar rutina y crear rutinas de recompensa inmutables (`isLocked`).
-

5. Capa de Presentación (`src/presentation/`)

Todo lo relacionado con la UI de React Native y Expo.

Configuración Visual

- `styles/theme.ts`: Archivo central de tokens de diseño. Define la paleta de colores oscuros, tipografía moderna, bordes y sombreados, garantizando homogeneidad en toda la app.

Navegación (`navigation/`)

- `AppNavigator.tsx`: Define el árbol de navegación. Usa un `TabNavigator` principal con cuatro ramas principales (Home, Ejercicios, Progreso, Perfil), cada una siendo un `StackNavigator` para navegación profunda. Escucha cambios de Auth para mostrar login o la app principal.

Pantallas (`screens/`)

Autenticación

- `auth/AuthScreen.tsx`: Pantalla inicial con pestañas para Iniciar Sesión o Registrarse. Diseño inmersivo de pantalla completa.

Principal

- `home/HomeScreen.tsx`: Dashboard principal. Muestra rutinas creadas, un widget circular con el XP y nivel actual, y un acordeón para visualizar/ocultar "Rutinas de Maestría".

Perfil

- `profile/ProfileScreen.tsx`: Muestra estadísticas globales, historial reciente y configuraciones. Incluye botones críticos de Cerrar Sesión y Eliminar Cuenta.

Árbol de Habilidades

- `skills/SkillsScreen.tsx`: Interfaz interactiva donde los nodos se conectan visualmente. Permite gastar XP para desbloquear ramas, revelando tips avanzados y nuevas plantillas de rutinas.

Gestión de Rutinas y Ejercicios

- `routine/RoutineEditScreen.tsx`: Interfaz compleja para editar una rutina en progreso. Permite añadir ejercicios, registrar series (peso y repeticiones) y finalizar el entrenamiento.
 - `routine/RoutineDetailScreen.tsx`: Vista de solo lectura para revisar el resumen de un entrenamiento finalizado o una plantilla.
 - `routine/ExerciseLibraryScreen.tsx`: Lista buscable y filtrable de todos los ejercicios disponibles. Permite marcar favoritos.
 - `routine/ExerciseDetailScreen.tsx`: Muestra cómo ejecutar un ejercicio, los músculos implicados y, fundamentalmente, el historial personal del usuario (1RM, volumen máximo) para ese ejercicio.
 - `routine/ExerciseCreateScreen.tsx`: Formulario para añadir ejercicios personalizados a la base de datos local.
-

6. Capa de Infraestructura (`src/infrastructure/`)

- `config/firebase.ts`: Inicializa la conexión con el SDK de Firebase utilizando las credenciales del proyecto. Exporta los servicios de `auth` y `db` (Firestore) para ser inyectados en los repositorios.